



南京大學
NANJING UNIVERSITY



PALM: Synergizing Program Analysis and LLMs to Enhance Rust Unit Test Coverage

Bei Chu, Yang Feng, Kui Liu, Hange Shi, Zifan Nan,
Zhaoqiang Guo and Baowen Xu

Software Security and Compiler Technology Lab
School of Computer Science, Nanjing University

Software Engineering Application Technology Lab, Huawei

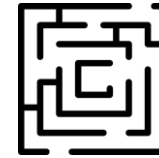
Generating High-Coverage Unit Tests is Challenging



Generating High-Coverage Unit Tests is Challenging



Traditional Methods (SBST, Concolic)

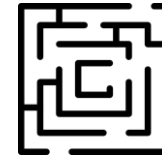


- Struggle with Rust's complexity (ownership, concurrency)
- Suffer from **Path Explosion** and **Environment Modeling** issues
- **Result:** Limited coverage for complex logic

Generating High-Coverage Unit Tests is Challenging



Traditional Methods (SBST, Concolic)



- Struggle with Rust's complexity (ownership, concurrency)
- Suffer from **Path Explosion** and **Environment Modeling** issues
- **Result:** Limited coverage for complex logic

Existing LLM Methods

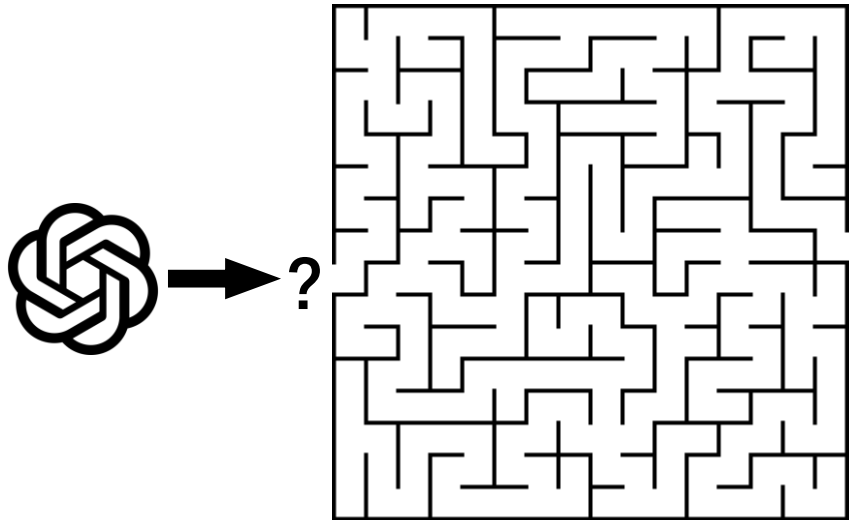


- Rely on **fixed, generic prompts**
- Lack specific guidance on internal logic
- **Result:** Low compilation rates & poor branch coverage

Our Insight: Guide the LLM with a Path-Specific Map

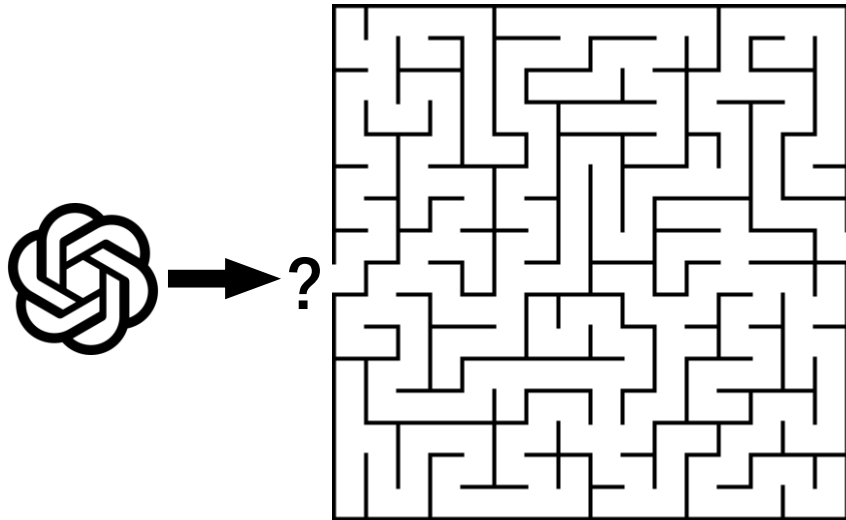


Our Insight: Guide the LLM with a Path-Specific Map

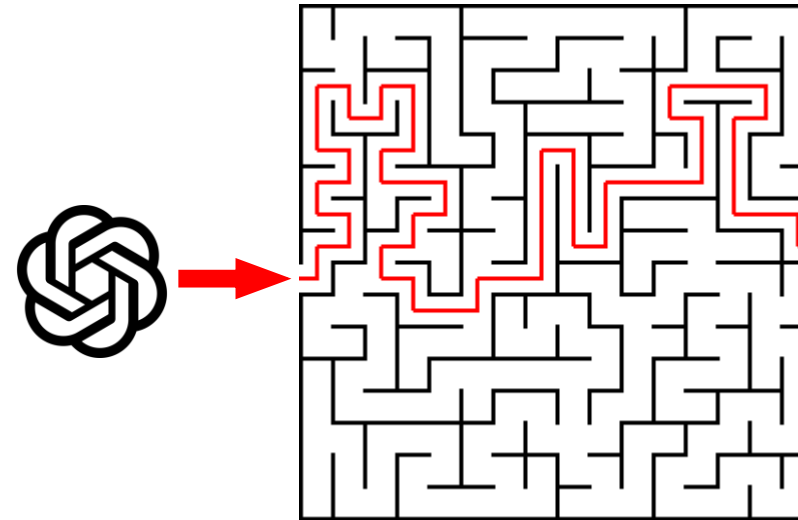


Common Approach: Ask the LLM to generate a whole test suite at once.

Our Insight: Guide the LLM with a Path-Specific Map



Common Approach: Ask the LLM to generate a whole test suite at once.

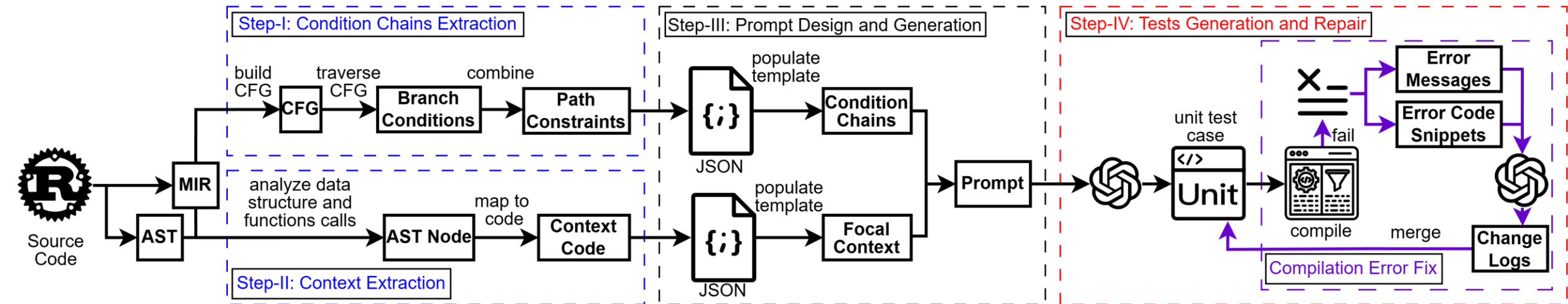


Our Approach: Guide the LLM with a specific mission for each execution path.

PALM: A Path-Guided, LLM-Powered Test Generator



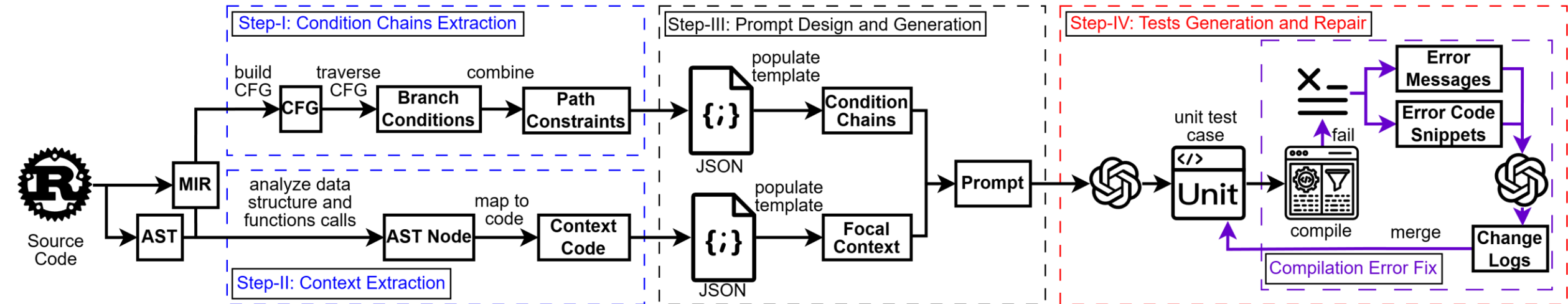
PALM: A Path-Guided, LLM-Powered Test Generator



PALM: A Path-Guided, LLM-Powered Test Generator

1. Path & Constraint Extraction

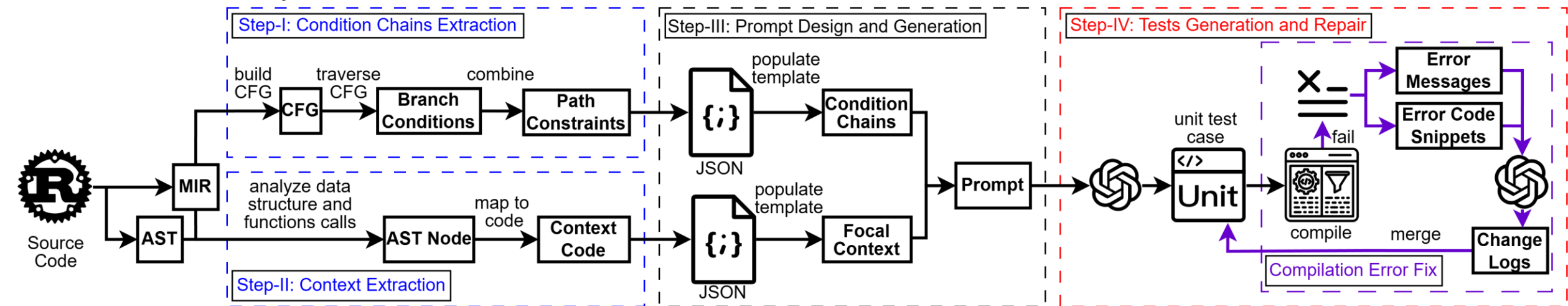
Analyze MIR/HIR to get precise condition chains.



PALM: A Path-Guided, LLM-Powered Test Generator

1. Path & Constraint Extraction

Analyze MIR/HIR to get precise condition chains.



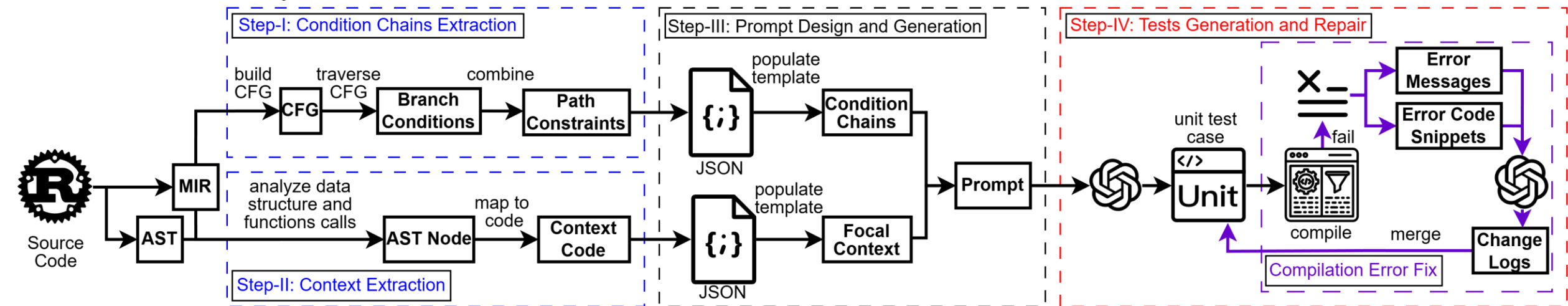
2. Context Extraction

Gather relevant struct/trait definitions.

PALM: A Path-Guided, LLM-Powered Test Generator

1. Path & Constraint Extraction

Analyze MIR/HIR to get precise condition chains.



2. Context Extraction

Gather relevant struct/trait definitions.

3. Path-Guided Prompting

Create a specific prompt for each path.

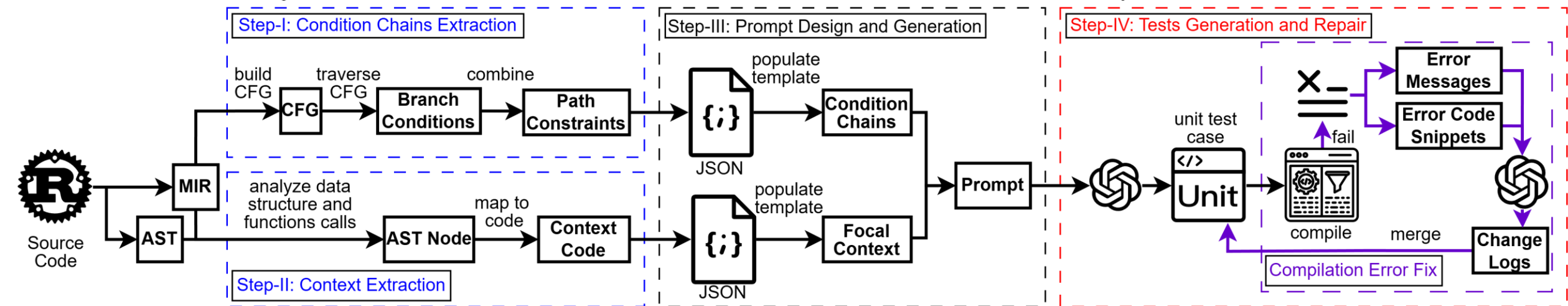
PALM: A Path-Guided, LLM-Powered Test Generator

1. Path & Constraint Extraction

Analyze MIR/HIR to get precise condition chains.

4. Iterative Repair Loop

Use compiler feedback to fix errors automatically.



2. Context Extraction

Gather relevant struct/trait definitions.

3. Path-Guided Prompting

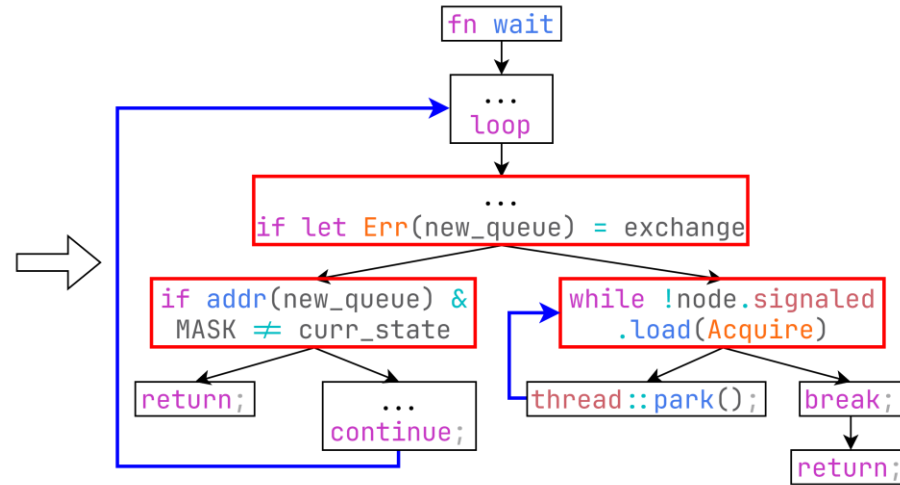
Create a specific prompt for each path.

Program Analysis using MIR/HIR



Program Analysis using MIR/HIR

```
fn wait(queue, mut curr_queue) {
  ...
  loop {
    ...
    if let Err(new_queue) = exchange {
      if addr(new_queue) &
        MASK != curr_state {
        return;
      }
      ...
      continue;
    }
    while !node.signaled.load(Acquire) {
      thread::park();
    }
    break;
  }
}
```

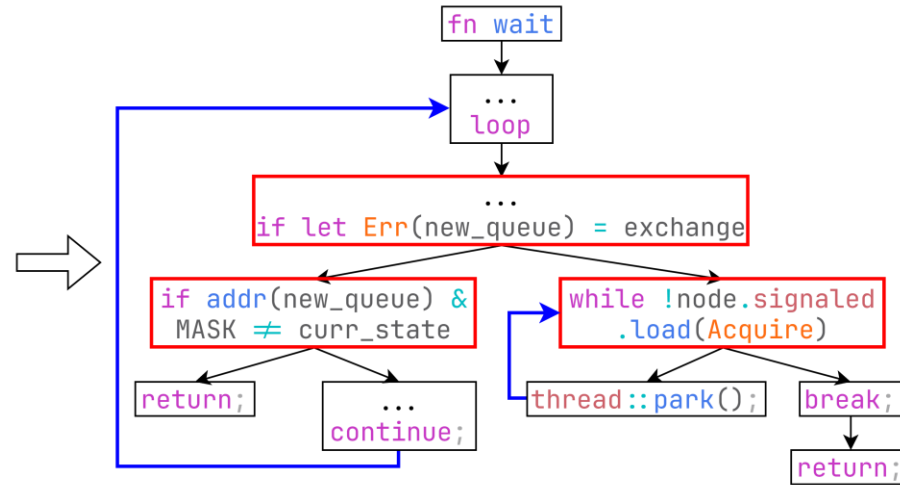


Condition Chains Extraction

- Analyze **MIR** for precise CFG →
- Traverse CFG to extract paths →
- Map back to **HIR** to recover source-level semantics →
- **Output:** Fine-grained **Condition Chains**

Program Analysis using MIR/HIR

```
fn wait(queue, mut curr_queue) {
    ...
    loop {
        ...
        if let Err(new_queue) = exchange {
            if addr(new_queue) &
                MASK != curr_state {
                return;
            }
            ...
            continue;
        }
        while !node.signaled.load(Acquire) {
            thread::park();
        }
        break;
    }
}
```



Condition Chains Extraction

- Analyze **MIR** for precise CFG →
- Traverse CFG to extract paths →
- Map back to **HIR** to recover source-level semantics →
- Output:** Fine-grained **Condition Chains**

<pre>use std::cell::{Cell, UnsafeCell}; ...</pre>	Uses
<pre>const INCOMPLETE: usize = 0x0; const RUNNING: usize = 0x1; ...</pre>	Consts
<pre>struct Waiter { thread: Cell<Option<Thread>>, signaled: AtomicBool, next: *mut Waiter, }</pre>	Structs
<pre>fn wait(queue, mut curr_queue) { ... }</pre>	Focal Method
<pre>mod strict { pub(crate) fn addr<T>(ptr: *mut T) → usize where T: Sized, { unsafe { core::mem::transmute(ptr) } } }</pre>	Other Methods

Context Extraction

- Identify all dependencies of the focal method:
 - Struct & Enum definitions
 - Trait implementations
 - Called functions
- Provide necessary context to ensure valid code generation.

Guided Generation and Automated Repair



Guided Generation and Automated Repair

As a software testing expert...

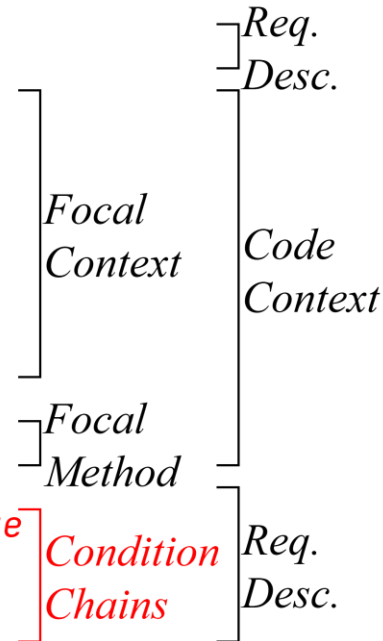
```
use std::{...};
struct Waiter {...}
fn wait(...) {...}
mod strict {
    pub(crate) fn addr<T>(...) → usize
    ...
}
```

The function to be tested is presented:

```
fn wait(...) {...}
```

Given the following constraints...:

```
// constraint: exchange matches Err(...) is true
// constraint: addr(...) & MASK ≠ ... is false
// constraint: ...
```



Path-Guided Prompt

Prompt =

- Task Instruction (“As a software testing expert...”)
- Focal Method + Context
- Target Condition Chain:
 - // constraint: exchange matches Err(...) is true
 - // constraint: strict::addr(...) & MASK != ... is false

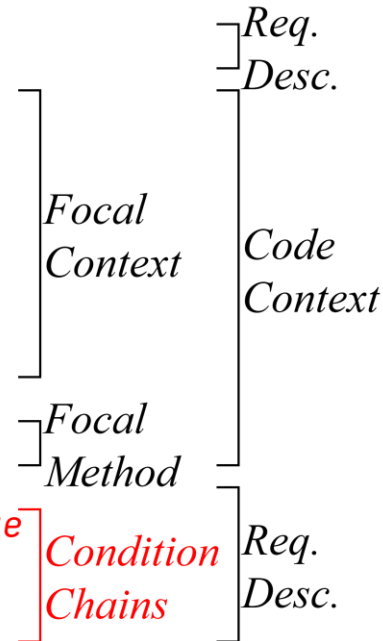
Guided Generation and Automated Repair

```
As a software testing expert...
use std::{...};
struct Waiter {...}
fn wait(...) {...}
mod strict {
    pub(crate) fn addr<T>(...) → usize
    ...
}
```

The function to be tested is presented:
fn wait(...) {...}

Given the following constraints...:

```
// constraint: exchange matches Err(...) is true
// constraint: addr(...) & MASK ≠ ... is false
// constraint: ...
```



Path-Guided Prompt

Prompt =

- Task Instruction (“As a software testing expert...”)
- Focal Method + Context
- Target Condition Chain:
 - // constraint: exchange matches Err(...) is true
 - // constraint: strict::addr(...) & MASK != ... is false

Error

```
#[test]
fn test_03() {
    ...
    wait(&queue, curr_queue);
    assert_eq!(queue.load(Ordering::Acquire)
               & STATE_MASK, COMPLETE);
}
```

Correct

```
#[test]
fn test_03() {
    ...
    wait(&queue, curr_queue);
    assert_eq!(queue.load(Ordering::Acquire)
               as usize & STATE_MASK, COMPLETE);
}
```

Automated Repair

Loop: Compile → Capture Error → Prompt LLM for Fix → Apply Patch

Before:load(...) & STATE_MASK (Type Error)

After:load(...) as usize & STATE_MASK (Fixed!)

Experimental Setup



Research Questions

- **RQ1: Effectiveness?**
(vs. Baselines)
- **RQ2: Ablation Study?**
(Impact of each component)
- **RQ3: Practical Usability?**
(Real-world PRs)

Research Questions

- **RQ1: Effectiveness?**
(vs. Baselines)
- **RQ2: Ablation Study?**
(Impact of each component)
- **RQ3: Practical Usability?**
(Real-world PRs)

Baselines

- **Classic:** RustyUnit (SBST), RuMono (Fuzzing)
- **LLM-based:** RUG, SymPrompt, ChatTester

Research Questions

- **RQ1: Effectiveness?**
(vs. Baselines)
- **RQ2: Ablation Study?**
(Impact of each component)
- **RQ3: Practical Usability?**
(Real-world PRs)

Baselines

- **Classic:** RustyUnit (SBST), RuMono (Fuzzing)
- **LLM-based:** RUG, SymPrompt, ChatTester

Subjects

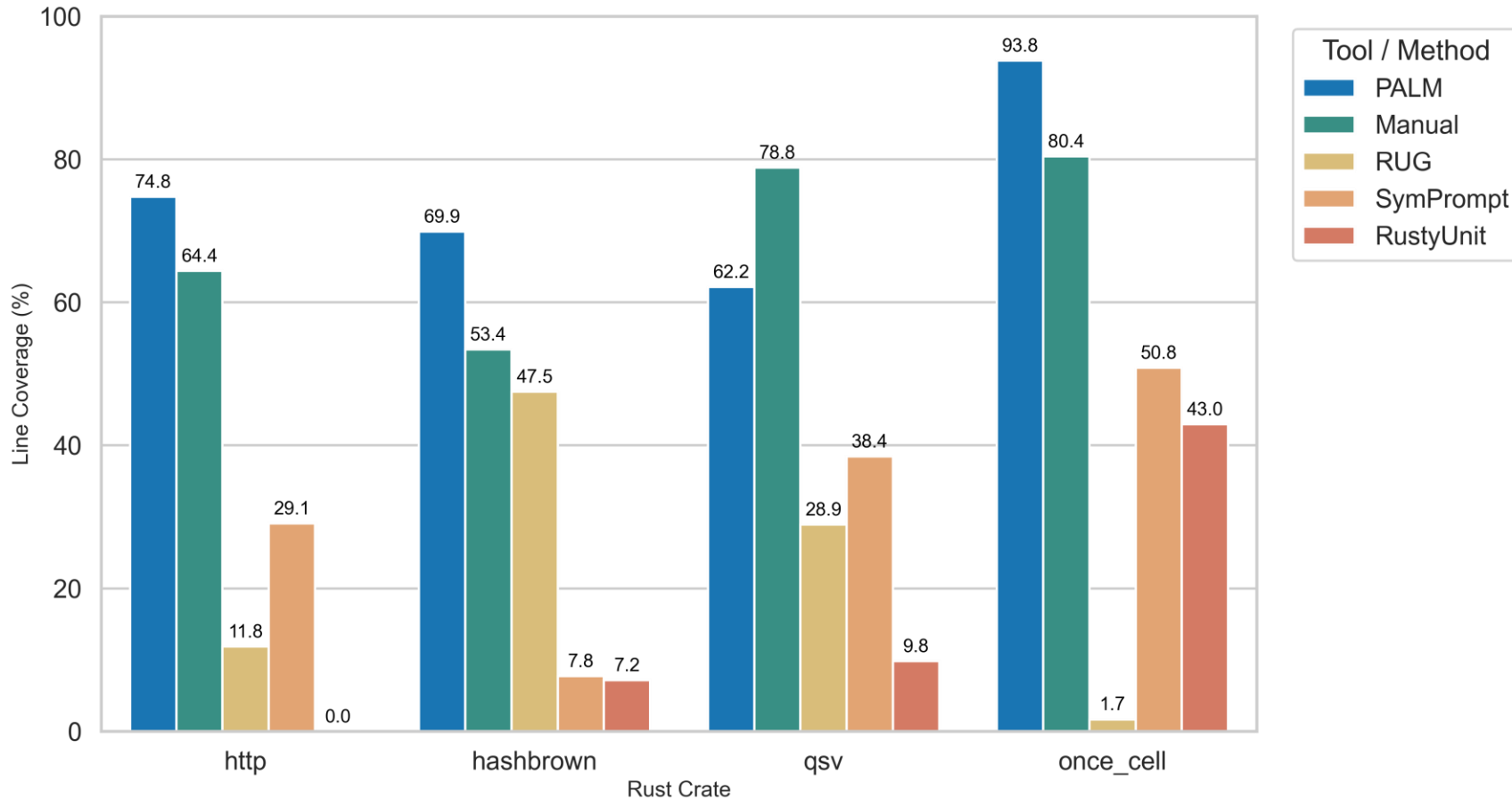
- **15** popular, mature Rust crates (e.g., serde, rand, regex, http, hashbrown)
- **4145** focal methods in total

PALM Significantly Outperforms Baselines in Coverage



PALM Significantly Outperforms Baselines in Coverage

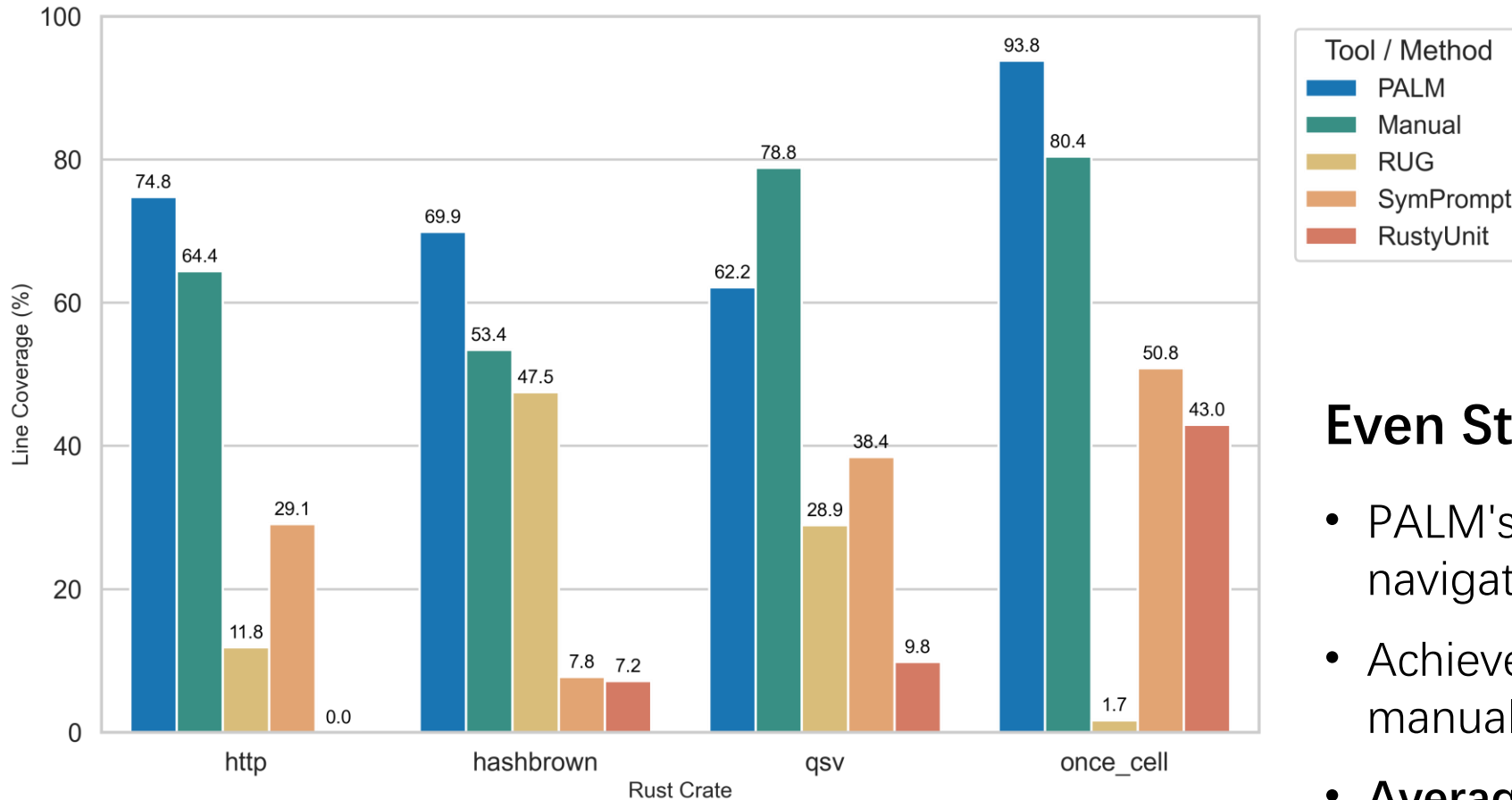
Line Coverage Comparison on Representative Crates



PALM's average line coverage (**72.30%**) is comparable to human effort (**70.94%**) and significantly outperforms other automated tools.

PALM Significantly Outperforms Baselines in Coverage

Line Coverage Comparison on Representative Crates



PALM's average line coverage (**72.30%**) is comparable to human effort (**70.94%**) and significantly outperforms other automated tools.

Even Stronger on Branch Coverage

- PALM's path-guided approach excels at navigating complex conditional logic.
- Achieved **higher branch coverage** than manual tests in **10 out of 15** projects.
- **Average Branch Coverage:**
 - **PALM: 69.5%**
 - Manual: 66.5%

It Works in Practice! High Acceptance Rate in Open-source Projects.





94.1%
Acceptance Rate

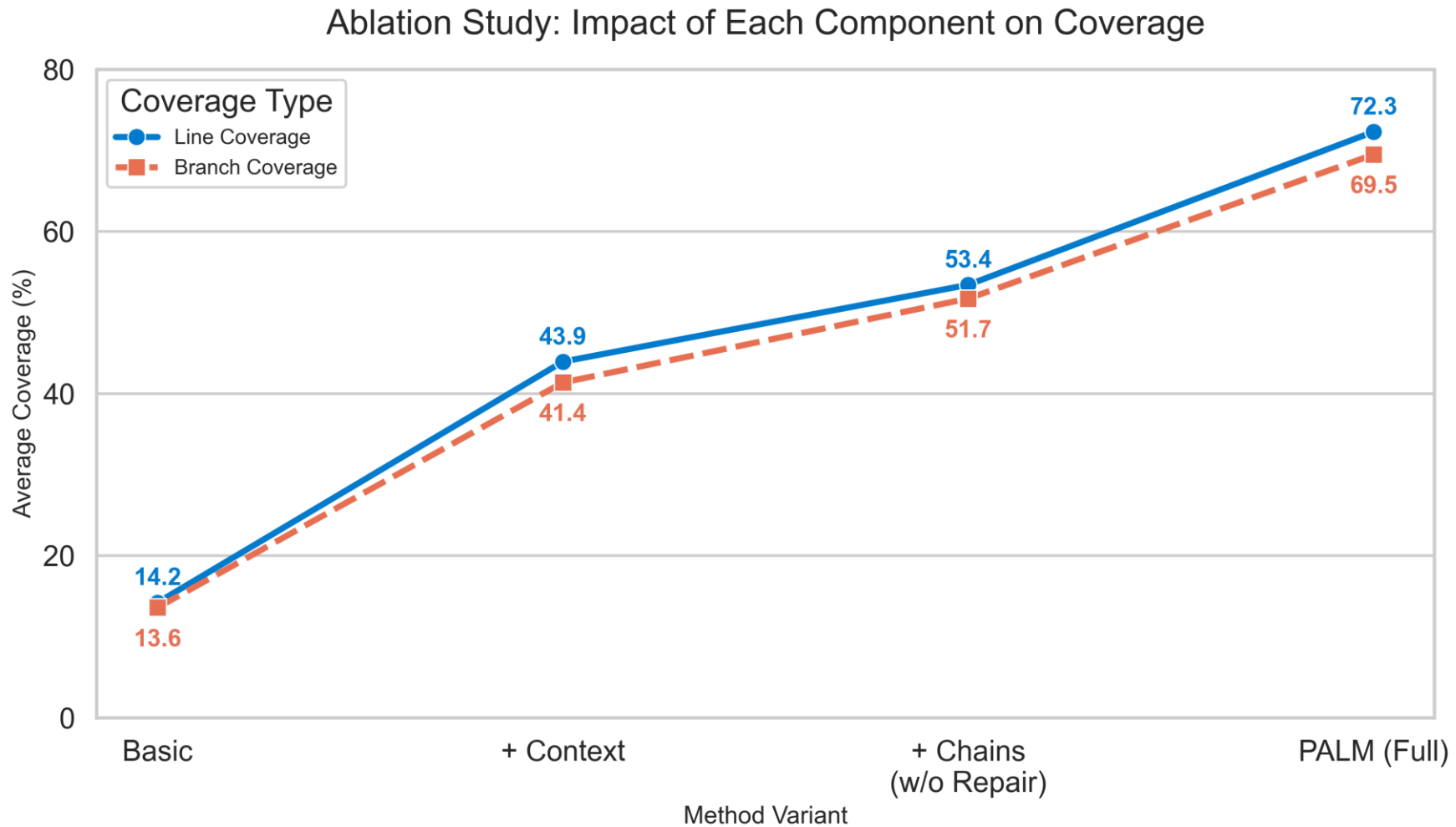
- Submitted **91** PALM-generated unit tests to 8 popular open-source projects.
- **80 tests were Accepted & Merged** by maintainers.
- Contributions to: hashbrown, once_cell, bytes, and more.

Among resolved submissions: 80 accepted / 85 resolved; 6 pending excluded.

Ablation Study: Path-Guidance and Repair are Crucial



Ablation Study: Path-Guidance and Repair are Crucial



Key Finding: Both **Path-Specific Guidance (Condition Chains)** and **Automated Repair** are crucial for achieving high performance.

Conclusion





- **PALM:** A novel approach synergizing program analysis and LLMs for Rust test generation.
- **Path-Guided Prompting:** Decompose the complex task into manageable steps, significantly improving coverage.
- **Iterative Repair:** Effectively fix compilation errors, making LLM-generated code practical and usable.
- **Proven Practicality:** Achieved coverage comparable to human effort and was validated by a **94.1%** pull request acceptance rate.

Acceptance rate: 80 accepted / 85 resolved; 6 pending excluded (see slide 28).

Thank You! Q&A

Email: beichu@smail.nju.edu.cn



Scan for paper